

MIAE 215 · Comprehensive Topic Guide (Part 5)

Program Architecture, Output Tracing & Algorithmic Patterns

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Course: Mechanical, Industrial & Aerospace Engineering (MIAE 215) · Fall 2026

Table of Contents

1. [The 7-Step Engineering Program Architecture \(The Universal Blueprint\)](#)
2. [C++ Data Type Modifiers & Memory Architecture](#)
3. [Systematic Table-Based Execution Tracing for Exam Output Questions](#)
4. [Statistical Vector Algorithms: Mean, Euclidean Norm & Sentinel Termination](#)
5. [Robust Menu Control & Defensive Input Validation](#)
6. [Array Extremum Search: The "Bigger and Better Deal" Algorithm](#)
7. [The C++ "Lego Box": Reusable Algorithmic Building Blocks](#)

1. The 7-Step Engineering Program Architecture (The Universal Blueprint)

Engineering students frequently confront five paralyzing obstacles during programming assignments and timed examinations:

1. *Where to start a program?*
2. *How to organize and structure code logic?*
3. *How to properly finish and terminate execution?*
4. *How to secure maximum partial marks on tests and midterms?*
5. *How to get unstuck when logic becomes tangled?*

To resolve these difficulties, the course establishes a deterministic **7-Step Structural Sequence**. Adhering to these steps sequentially transforms abstract engineering problem statements into robust, clean C++ code.

```
[1. DECLARE] -> [2. INITIALIZE] -> [3. INPUT] -> [4. CONTROL STATEMENTS] -> [5. EXPRESSIONS] -> [6. OUTPUT] ->
```

1.1 Step 1: DECLARE

- **Objective:** Identify all state variables, counters, mathematical constants, and data buffers directly from the problem statement.
- **Rule:** Declare **ALL** variables at the very beginning of the function or program block.
- **Best Practice:** Group by type (`int`, `double`, arrays), provide meaningful names, and leave dedicated headroom at the top of `main()` to declare additional auxiliary variables as development proceeds.

1.2 Step 2: INITIALIZE

- **Objective:** Assign well-defined initial values to eliminate uninitialized memory garbage.
- **Rule:** In C++, local primitive variables are allocated on the runtime stack without zeroing; reading an uninitialized variable yields arbitrary bit patterns (**garbage in** $\implies$ **garbage out**).
- **Rule:** Initialize accumulators (`sum = 0.0`), product bases (`prod = 1.0`), counters (`count = 0`), and tolerance bounds (`eps = 1e-7`) immediately following declaration.

1.3 Step 3: INPUT

- **Objective:** Ingest data provided by external sources (keyboard streams, ASCII files, Arduino analog/digital sensors, joystick buffers).
- **Exam Strategy:** If an exam problem explicitly states "*check for errors*", write defensive boundary verification logic. If the problem does not specify error checking, assume inputs conform to specifications. In real-world engineering, inputs must always be validated.

1.4 Step 4: CONTROL STATEMENTS ("The C++ Lego Framework")

- **Objective:** Erect the structural skeleton and execution branching of the program before filling in algebraic formulas.
- **The Concept of C++ Legos:** Every complex program is assembled from standard control blocks:
 - *Counting iteration:* `for (i = 0; i < n; i++)`
 - *Indefinite sensory / sentinel loops:* `while (condition)` or `while (1)` with `break`
 - *Threshold classification:* `if-else` cascades and ladders
- **Execution Strategy:** Write control statements leaving empty blanks for calculations, establishing a clear structural template that can be populated methodically.

1.5 Step 5: EXPRESSIONS

- **Objective:** Compute required physical and mathematical variables by placing equations inside the control structures erected in Step 4.
- **Rule:** Populate the skeleton with operators, trigonometric evaluations, and algebraic updates (e.g., `sum += A[i]*A[i]`).

1.6 Step 6: OUTPUT

- **Objective:** Transmit computed answers to external recipients (console streams `cout` , data loggers, graphical LCDs, or motor actuators).
- **Formatting:** Label all outputs explicitly with engineering units and descriptive identifiers (e.g., `cout << "\nA_ave = " << A_ave;`).

1.7 Step 7: DEBUG & TEST

- **Compile-Time vs. Run-Time Errors:** Syntax and type mismatch errors are caught by the compiler. Run-time errors (segmentation faults, infinite loops, logic flaws) require systematic diagnostic testing.
 - **Verification:** Test the algorithm against boundary values: $N = 0$, $N = 1$, extreme negative inputs, and boundary thresholds.
-

2. C++ Data Type Modifiers & Memory Architecture

Type modifiers alter the fundamental bit width, dynamic range, and sign interpretation of standard C++ types.

2.1 Integer Type Modifiers

Type Specification	Typical Bit Width	Signed / Unsigned	Minimum Value	Maximum Value
short int (short)	16 bits (2 bytes)	Signed (Two's Compl.)	$-32,768 (-2^{15})$	$+32,767 (2^{15} - 1)$
unsigned short	16 bits (2 bytes)	Unsigned	0	$+65,535 (2^{16} - 1)$
int	32 bits (4 bytes)	Signed	$-2,147,483,648 (-2^{31})$	$+2,147,483,647 (2^{31} - 1)$
unsigned int	32 bits (4 bytes)	Unsigned	0	$+4,294,967,295 (2^{32} - 1)$
long int (long)	32 or 64 bits	Signed	Typically -2^{31}	Typically $+2^{31} - 1$
unsigned long	32 or 64 bits	Unsigned	0	Typically $+2^{32} - 1$
long long int	64 bits (8 bytes)	Signed	$-2^{63} \approx -9.22 \times 10^{18}$	$+2^{63} - 1 \approx +9.22 \times 10^{18}$
unsigned long long	64 bits (8 bytes)	Unsigned	0	$+2^{64} - 1 \approx +1.84 \times 10^{19}$

2.2 Two's Complement & Unsigned Wraparound

- **Signed Representation:** The most significant bit (MSB) acts as the sign flag (0 = positive, 1 = negative). Negative integers are stored as the two's complement: invert all bits and add 1.
- **Unsigned Overflow:** When an unsigned short exceeds 65535, it silently wraps modulo 2^{16} :

$$65535 + 1 \longrightarrow 0$$

- **Engineering Principle:** Never use unsigned integers for variables that can legitimately decrease past zero (e.g., loop countdown counters for `unsigned int i = 5; i >= 0; i--`) create an **infinite loop** because `i` is always ≥ 0).

2.3 The const Modifier

The `const` keyword specifies an immutable read-only variable enforced at compile time:

```
const double PI = 3.141592653589793;
const int MAX_SAMPLES = 200;
```

Attempting to modify `PI = 3.14;` triggers a compile-time error, preventing accidental corruption of physical engineering parameters.

3. Systematic Table-Based Execution Tracing for Exam Output Questions

On MIAE 215 examinations and quizzes, students are explicitly required to determine program outputs by constructing a formal **Execution Trace Table**. Writing down the final output without the supporting step table results in loss of marks.

3.1 The 3-Step Trace Procedure

1. **Label Lines Sequentially:** Assign an integer label to every sequential statement. In a `for` loop, assign distinct numbers to each of its three expressions: $\text{\texttt{\text{for}(\underbrace{i=0}_{\text{Expr 1 (Init)}}; \underbrace{i<=3}_{\text{Expr 2 (Condition)}}; \underbrace{i++}_{\text{Expr 3 (Increment)}})}$
2. **Construct the Variable State Table:** Create columns for Line Number, each active variable (`i`, `j`, `x`, `y`, array slots), condition evaluation (`true / false`), and Console Output.
3. **Trace Step-by-Step:** Execute instructions in exact CPU clock order. Record variable mutations at each line.

3.2 Worked Exam Trace 1: The Linear Accumulation Loop

```
int i, j;           // Line 1
double x, y;        // Line 2
j = -1;             // Line 3
x = 0.0;            // Line 4
y = 2.5;            // Line 5
for (i = 0; i <= 3; i++) { // Line 6 (init), Line 7 (test), Line 11 (step)
    x += y;          // Line 8
    j *= -1;         // Line 9
    cout << j << "   " << x << "
"; // Line 10
}
cout << "
i = " << i;        // Line 12
```

Complete Execution State Table:

Line	Execution Event / Description	i	j	x	y	Output Buffer
1-2	Declarations	G	G	G	G	(none)
3	j = -1	G	-1	G	G	(none)
4	x = 0.0	G	-1	0.0	G	(none)
5	y = 2.5	G	-1	0.0	2.5	(none)
6	Loop Initialization: i = 0	0	-1	0.0	2.5	(none)
7	Condition: 0 <= 3 $\implies$ True	0	-1	0.0	2.5	(none)
8	x += y $\implies$ 0.0 + 2.5	0	-1	2.5	2.5	(none)
9	j *= -1 $\implies$ (-1)(-1)	0	1	2.5	2.5	(none)
10	cout << j << " " << x	0	1	2.5	2.5	1 2.5
11	Loop Increment: i++ $\implies$ 0 + 1	1	1	2.5	2.5	(none)
7	Condition: 1 <= 3 $\implies$ True	1	1	2.5	2.5	(none)
8	x += y $\implies$ 2.5 + 2.5	1	1	5.0	2.5	(none)
9	j *= -1 $\implies$ (1)(-1)	1	-1	5.0	2.5	(none)
10	cout << j << " " << x	1	-1	5.0	2.5	-1 5
11	Loop Increment: i++ $\implies$ 1 + 1	2	-1	5.0	2.5	(none)
7	Condition: 2 <= 3 $\implies$ True	2	-1	5.0	2.5	(none)
8	x += y $\implies$ 5.0 + 2.5	2	-1	7.5	2.5	(none)
9	j *= -1 $\implies$ (-1)(-1)	2	1	7.5	2.5	(none)
10	cout << j << " " << x	2	1	7.5	2.5	1 7.5
11	Loop Increment: i++ $\implies$ 2 + 1	3	1	7.5	2.5	(none)
7	Condition: 3 <= 3 $\implies$ True	3	1	7.5	2.5	(none)
8	x += y $\implies$ 7.5 + 2.5	3	1	10.0	2.5	(none)
9	j *= -1 $\implies$ (1)(-1)	3	-1	10.0	2.5	(none)
10	cout << j << " " << x	3	-1	10.0	2.5	-1 10
11	Loop Increment: i++ $\implies$ 3 + 1	4	-1	10.0	2.5	(none)
7	Condition: 4 <= 3 $\implies$ False	4	-1	10.0	2.5	(loop terminates)
12	`cout << "					
i = " << i`	4	-1	10.0	2.5	i = 4	

Final Program Output:

1	2.5
-1	5
1	7.5
-1	10

i = 4

(Notice critically: after loop termination, `i` has reached 4, NOT 3. This is a primary teacher trap on examinations).

3.3 Worked Exam Trace 2: Array Mutations with `break` and `continue`

```
int A[5]; // Line 1
x = 1.5; // Line 2
i = 0; // Line 3
while (x < 35) { // Line 4
    i++; // Line 5
    x = 2 * x + 1; // Line 6
    if ((x - i*i*i) < 0) { // Line 7
        break; // Line 8
    }
    if (x > 15) { // Line 9
        continue; // Line 10
    }
    A[i] = 2 * i; // Line 11
}
cout << "
x = " << x; // Line 12
cout << "
i = " << i; // Line 13
cout << "
A[2] - A[0] = " << A[2] - A[0]; // Line 14
```

Complete Execution State Table:

Line	Execution Event / Description	i	x	A[0]	A[1]	A[2]	A[3]	A[4]	Test Result
1-3	Init: $x = 1.5, i = 0$	0	1.5	G	G	G	G	G	-
4	Loop Test: $1.5 < 35$	0	1.5	G	G	G	G	G	True
5	$i++ \Rightarrow 0 + 1$	1	1.5	G	G	G	G	G	-
6	$x = 2(1.5) + 1 \Rightarrow 4.0$	1	4.0	G	G	G	G	G	-
7	$x - i^3 \Rightarrow 4 - 1 = 3 < 0?$	1	4.0	G	G	G	G	G	False
9	$x > 15 \Rightarrow 4.0 > 15?$	1	4.0	G	G	G	G	G	False
11	$A[1] = 2(1) = 2$	1	4.0	G	2	G	G	G	-
4	Loop Test: $4.0 < 35$	1	4.0	G	2	G	G	G	True
5	$i++ \Rightarrow 1 + 1$	2	4.0	G	2	G	G	G	-
6	$x = 2(4.0) + 1 \Rightarrow 9.0$	2	9.0	G	2	G	G	G	-
7	$x - i^3 \Rightarrow 9 - 8 = 1 < 0?$	2	9.0	G	2	G	G	G	False
9	$x > 15 \Rightarrow 9.0 > 15?$	2	9.0	G	2	G	G	G	False
11	$A[2] = 2(2) = 4$	2	9.0	G	2	4	G	G	-
4	Loop Test: $9.0 < 35$	2	9.0	G	2	4	G	G	True
5	$i++ \Rightarrow 2 + 1$	3	9.0	G	2	4	G	G	-
6	$x = 2(9.0) + 1 \Rightarrow 19.0$	3	19.0	G	2	4	G	G	-
7	$x - i^3 \Rightarrow 19 - 27 = -8 < 0?$	3	19.0	G	2	4	G	G	True $\Rightarrow$ break !
8	Immediate exit to Line 12	3	19.0	G	2	4	G	G	-
12	`cout << "								
x = "<< x`	3	19.0	G	2	4	G	G	Output: x = 19	
13	`cout << "								
i = "<< i`	3	19.0	G	2	4	G	G	Output: i = 3	
14	cout << A[2] - A[0]	3	19.0	G	2	4	G	G	4 – Garbage = Undefined / Garbage

Key Pedagogical Takeaways:

1. Notice that `continue` (Line 10) was never triggered because the loop was abruptly terminated by `break` when $x - i^3 = -8 < 0$.
2. `A[0]` was **never assigned** a value. Evaluating `A[2] - A[0]` attempts arithmetic on an uninitialized memory slot (Garbage). On tests, students must identify that `A[0]` contains garbage (G), so `A[2] - A[0] = G` or undefined!

4. Statistical Vector Algorithms: Mean, Euclidean Norm & Sentinel Termination

Engineering data analysis regularly requires streaming dynamic sensor telemetry into a fixed array, terminating when a sentinel value is received, and extracting statistical metrics.

4.1 The Sentinel Stream Pattern

When data count is not known a priori:

```
const int N_MAX = 200;
double A[N_MAX];
int ndata = 0;

for (int i = 0; i < N_MAX; i++) {
    cout << "Input A[" << i << "]? ";
    cin >> A[i];
    if (A[i] < -1.0) { // Sentinel condition: stop on any value < -1
        ndata = i;
        break;
    }
}
```

4.2 Statistical Metrics: Mean & Vector Norm

1. **Sample Mean (Average):**

$$\bar{A} = \frac{1}{N} \sum_{i=0}^{N-1} A_i$$

2. **Euclidean Vector Norm (Magnitude):** $\|A\|_2 = \sqrt{\sum_{i=0}^{N-1} A_i^2}$

4.3 Computational Efficiency: `A[i]*A[i]` vs. `pow(A[i], 2)`

In C++, calling `pow(x, 2.0)` involves function call overhead and general logarithmic/exponential hardware implementations ($e^{2 \ln x}$). Writing `A[i] * A[i]` produces a single inline floating-point multiplication instruction (`FMUL`), running up to $10\times$ to $20\times$ **faster**.

4.4 Multi-Condition Filter with Floating-Point Epsilon

To count elements satisfying $1 < A_i < 10$ OR $A_i == 0$:

```

const double EPS = 1e-7;
int count = 0;

for (int i = 0; i < ndata; i++) {
    // Floating point zero test must use abs(A[i]) < EPS
    if ((1.0 < A[i] && A[i] < 10.0) || abs(A[i]) < EPS) {
        count++;
    }
}

```

5. Robust Menu Control & Defensive Input Validation

Interactive control interfaces must trap invalid user inputs without crashing.

5.1 The Infinite Loop Validation Pattern

```

int k;
while (1) {
    cout << "\nEnter menu choice (1 to 3): ";
    cin >> k;

    // Validate range
    if (1 <= k && k <= 3) {
        break; // Valid input received: exit guard loop
    } else {
        cout << "Input is out of range! Please try again.\n";
    }
}

```

5.2 Inverted Condition via De Morgan's Laws

The boundary can equivalently be checked by testing for out-of-range inputs:

$$\text{Out of Range} \iff (k < 1) \vee (k > 3)$$

```

if (k < 1 || k > 3) {
    cout << "Input is out of range";
} else {
    break;
}

```

5.3 Multi-Way Branching: `if-else` Ladder

```
if (k == 1) {
    cout << "Option 1: Calibrate Load Cell\n";
} else if (k == 2) {
    cout << "Option 2: Zero Stepper Motor\n";
} else if (k == 3) {
    cout << "Option 3: Run Trajectory Automation\n";
} else {
    cout << "Error: Unhandled branch k = " << k << "\n";
}
```

6. Array Extremum Search: The "Bigger and Better Deal" Algorithm

Finding the maximum or minimum value in an array is a foundational algorithmic pattern.

6.1 The Two Initialization Philosophies

1. The Arbitrary Small Value Approach:

```
double max_A = -1e10; // Initialize to large negative constant
for (int i = 0; i < n; i++) {
    if (A[i] > max_A) {
        max_A = A[i];
    }
}
```

Limitation: Fails if all input values are smaller than -10^{10} (e.g., deep cryogenic or astrophysical data).

2. The First-Element Approach (Mathematically Rigorous & Universal):

```
double max_A = A[0]; // Candidate is guaranteed to be a valid member of array
for (int i = 1; i < n; i++) { // Loop starts at index 1!
    if (A[i] > max_A) {
        max_A = A[i];
    }
}
```

Advantage: Works for all possible values of $A_i \in (-\infty, +\infty)$ without requiring prior domain assumptions.

7. The C++ "Lego Box": Reusable Algorithmic Building Blocks

Engineers do not write code from scratch for every assignment; they assemble pre-verified "Lego blocks":

Algorithmic Lego Block	Syntax Pattern	Primary Use Case
1D Array Traversal	<code>for (int i = 0; i < n; i++)</code>	Sequential access, scalar accumulation
Sentinel Input Stream	<code>while (cin >> val && val >= 0)</code>	Unknown quantity of sensory stream data
Accumulator	<code>sum += A[i];</code>	Centroid, energy, statistical mean
Max / Min Extremum	<code>if (A[i] > max_val) max_val = A[i];</code>	Peak stress, maximum temperature, failure analysis
Filter Counter	<code>if (condition) count++;</code>	Anomaly detection, quality control binning
Defensive Menu Loop	<code>while (1) { cin >> k; if (valid) break; }</code>	Robust embedded systems UI